

Database Sharding

Database Sharding is a horizontal partitioning strategy that splits a large database into smaller, more manageable pieces called "shards," with each shard stored on a separate database server.

Core Concept

Instead of storing all data in one massive database, you split it across multiple databases. Each shard contains a subset of the total data, and together all shards comprise the complete dataset.

Why Shard?

Performance limits Single database can't handle the query load—too many concurrent requests.

Storage limits Data grows beyond what one server can store efficiently.

Geographic distribution Place data closer to users in different regions for lower latency.

Cost optimization Scale horizontally with commodity hardware instead of expensive vertical scaling.

Availability Isolate failures—one shard failing doesn't bring down the entire system.

Sharding Strategies

1. Range-Based Sharding : Divide data based on ranges of a key value

Example - Date ranges:

```
Shard 1: Orders from 2020
Shard 2: Orders from 2021
Shard 3: Orders from 2022
Shard 4: Orders from 2023-2024
```

Advantages:

- Simple to implement and understand
- Range queries are efficient (all data on one shard)
- Easy to add new ranges

Disadvantages:

- Can lead to uneven data distribution (hotspots)
- Newer data often accessed more (recent orders) → one shard overloaded
- Manual rebalancing needed when ranges fill up

Best for:

- Time-series data
- Sequential IDs
- When range queries are common

2. Hash-Based Sharding : Apply hash function to shard key, use result to determine which shard.

Example:

```
shard = hash(user_id) % number_of_shards

user_id: 12345
hash(12345) = 789234
789234 % 4 = 2
→ Store in Shard 2
```

Advantages:

- Even data distribution (if good hash function)
- Automatic balancing
- No hotspots from sequential keys

Disadvantages:

- Range queries require querying all shards
- Adding/removing shards requires resharding (many records move)
- Cannot easily query related data together

Best for:

- Even distribution is priority
- Mostly single-key lookups
- User-based data (user profiles, sessions)

3. Directory-Based Sharding (Lookup Table) : Maintain a lookup table that maps each key to its shard location.

Example:

```
Lookup Table:
user_id | shard_location
-----|-----
1       | shard_1
2       | shard_3
3       | shard_2
...     | ...
1000000 | shard_4

Query for user_id=2:
1. Check lookup table → shard_3
2. Query shard_3 for user_id=2
```

Advantages:

- Most flexible approach
- Easy to rebalance (just update lookup table)
- Can use any logic for assignment
- Can move individual records between shards

Disadvantages:

- Lookup table becomes a bottleneck and single point of failure
- Extra hop for every query (lookup first, then query shard)
- Lookup table itself needs to scale and be highly available

Best for:

- Complex sharding logic
- Frequently changing shard assignments
- When you need maximum flexibility

4. Geographic/Location-Based Sharding : Shard based on user location or data origin.**Example:**

```
Shard 1 (US-East): Users in New York, Boston, Miami
Shard 2 (US-West): Users in LA, San Francisco, Seattle
Shard 3 (EU): Users in London, Paris, Berlin
Shard 4 (APAC): Users in Tokyo, Singapore, Sydney
```

Advantages:

- Lower latency (data close to users)
- Regulatory compliance (data residency requirements)
- Natural boundaries reduce cross-shard queries

Disadvantages:

- Uneven distribution (more users in some regions)
- User movement requires data migration
- Cross-region queries are slow

Best for:

- Global applications
- Data residency requirements (GDPR, etc.)
- Latency-sensitive applications

5. Entity-Based/Service-Based Sharding : Different entities or microservices have their own sharded databases.**Example:**

Users Service:

- Shard 1: User profiles 1-1M
- Shard 2: User profiles 1M-2M

Orders Service:

- Shard 1: Orders for users 1-1M
- Shard 2: Orders for users 1M-2M

Products Service:

- Shard 1: Products A-M
- Shard 2: Products N-Z

Advantages:

- Aligns with microservices architecture
- Different entities can use different sharding strategies
- Failure isolation

Disadvantages:

- Complex cross-service queries
- Eventual consistency challenges
- Duplicate data across services

Best for:

- Microservices architectures
- Large-scale systems with distinct domains

Choosing a Shard Key

The shard key is **the most critical decision** in sharding. It determines data distribution and query patterns.

Good shard key characteristics:

High cardinality Many distinct values to distribute across shards.

Good: user_id (millions of unique values) **Bad:** gender (only 2-3 values)

Even distribution Keys spread data uniformly across shards.

Good: hash(user_id) **Bad:** created_at (recent dates get all new data)

Query alignment Most queries include the shard key.

Good: If queries are "get user data for user_id=X", use user_id as shard key

Bad: If queries are "get all orders in last week", user_id shard key requires querying all shards

Immutable Key doesn't change (or changes rarely).

Good: user_id (never changes) **Bad:** user_status (changes frequently)

Common shard keys:

- user_id (user-centric apps)
- tenant_id (multi-tenant SaaS)
- customer_id (B2B platforms)
- device_id (IoT applications)
- session_id (session data)

Sharding Architecture Components

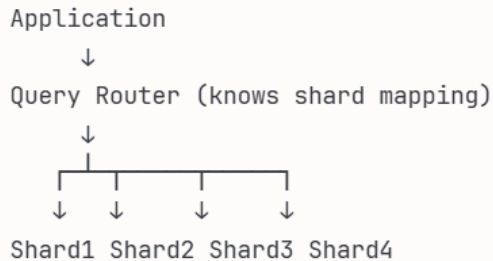
Shard (Database Instance) Individual database server holding a subset of data.

Router/Proxy Routes queries to the correct shard(s) based on shard key.

Config Server Stores shard mapping metadata (which shard has which data).

Application Logic May contain shard-aware code to route queries.

Example architecture:



Query Patterns with Sharding

Single-Shard Query

Query includes shard key:

sql

```
SELECT * FROM users WHERE user_id = 12345;
```

Router logic:

1. Determine shard: $\text{hash}(12345) \% 4 = \text{Shard } 2$
2. Query only Shard 2
3. Return results

Performance: FAST (one shard only)

Cross-Shard Query (Scatter-Gather)

Query doesn't include shard key:

sql

```
SELECT * FROM users WHERE email = 'alice@example.com';
```

Router logic:

1. Don't know which shard has this email
2. Query ALL shards
3. Combine results from all shards
4. Return merged results

Performance: SLOW (all shards queried)

Joins Across Shards

Problem:

```
sql
-- Users sharded by user_id
-- Orders sharded by user_id
SELECT users.name, orders.product
FROM users
JOIN orders ON users.id = orders.user_id
WHERE users.city = 'New York';
```

Issue: city filter requires **all user** shards, **then** need orders **from** multiple shards

Solutions:

- **Denormalize:** Include user info in orders table
- **Application-level joins:** Fetch from shards separately, join in app
- **Co-locate related data:** Shard users and orders by same key to same shards

Challenges of sharding ⚠

Sharding is powerful but **not free**.

- ✗ Cross-shard joins are hard
- ✗ Transactions across shards are complex
- ✗ Rebalancing data is painful
- ✗ Application logic becomes more complex

Real-world examples

- **MongoDB:** Built-in sharding support
- **MySQL:** Application-level sharding
- **Cassandra:** Sharding is native (via partition keys)
- **Twitter / Instagram:** User-based sharding

Mental Model : Sharding=splitting data *across* many DBs &
Partitioning=splitting data *inside* one DB